

LLM-инженер

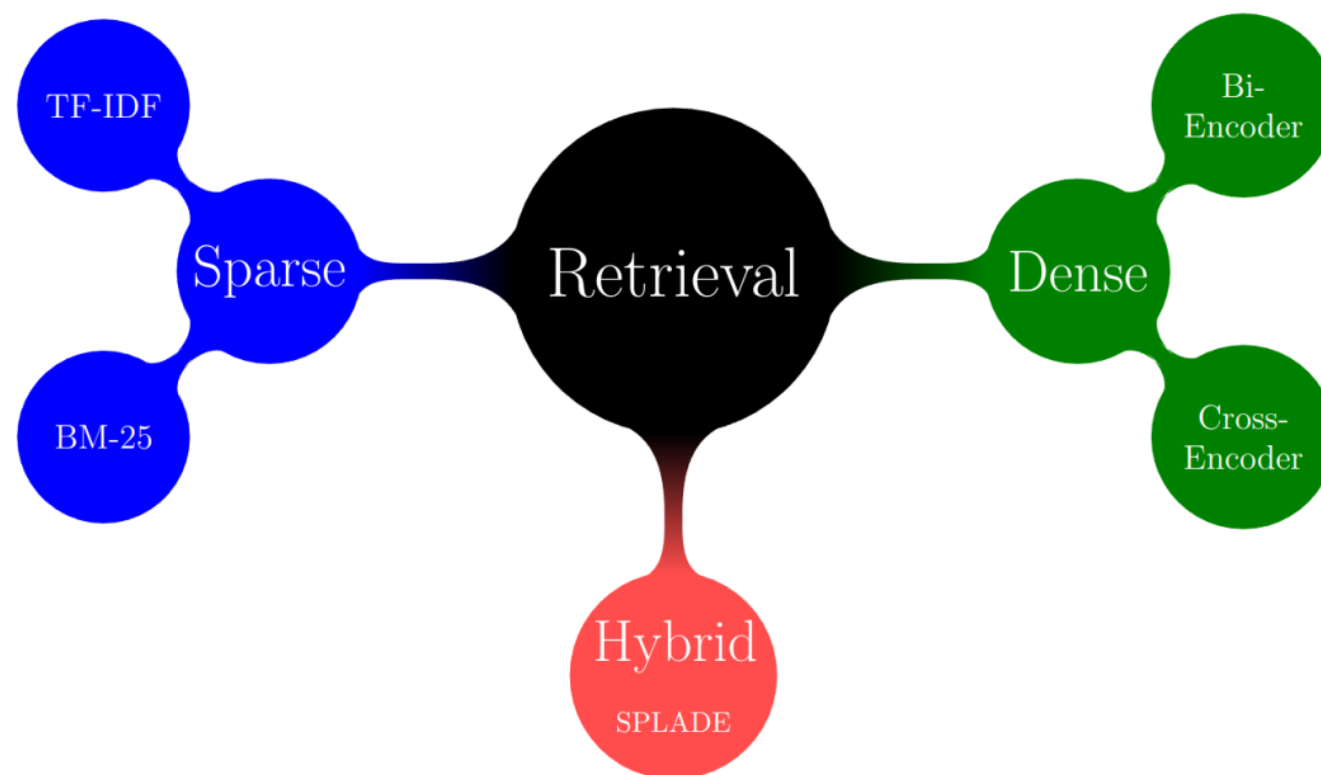
online course ✨

модуль #3

лекция: информационный поиск

Информационный поиск

Информационный поиск



Sparse-подходы

Разреженный лексический) поиск – поиск по совпадениям токенов с весами (tf-idf/BM25/...).

- Сильные стороны: редкие термины, точные цитаты, код/артефакты.
- Слабые стороны: парафразы, мультязычность, синонимия.

BM25

$$\text{score}(D, Q) = \underbrace{\sum_{i=1}^n}_{\text{Суммируем оценку каждого элемента запроса}} \underbrace{\ln\left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} + 1\right)}_{\text{Обратная частотность документа}} \cdot \underbrace{\frac{\overbrace{f(q_i, D) \cdot (k_1 + 1)}^{\text{Частотность элемента в документе}}}{f(q_i, D) + k_1 \cdot \underbrace{(1 - b + b \cdot \frac{|D|}{\text{avgdl}})}_{\text{Нормализация длины документа}}}}_{\text{Нормализация длины документа}}$$

- N — общее количество элементов в коллекции
- $n(q_i)$ — количество документов, содержащих элемент запроса
- Следовательно, $N - n(q_i)$ — это количество документов, *не содержащих* элемент запроса
- $f(q_i, D)$ — частотность запроса в документе
- k_1 — это регулировочный параметр term frequency, обычно имеющий значение от 1.2 до 2 (ниже — жёстче на повторения, выше — допускает спам терминами)
- $|D|$ — это длина документа
- avgdl — средняя длина документа в нашей коллекции
- b — ещё один регулировочный параметр, управляющий тем, насколько мы выполняем нормализацию по длине документа

Когда базовый BM25 «не тянет»

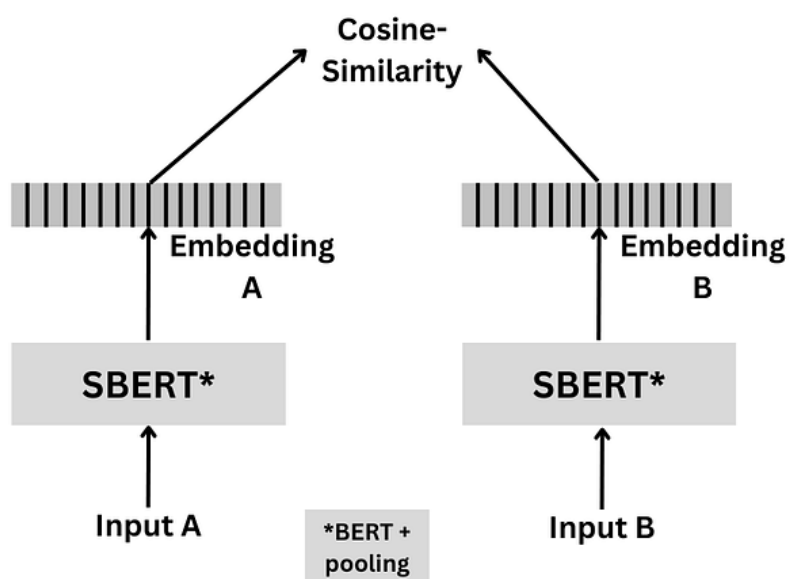
- **Обнуление вклада редких терминов** в очень длинных документах – может помочь BM25+. Добавляет маленькое смещение в TF, позволяя редким словам в длинных документах иметь ненулевой вес.
- **Переоценка частых токенов**, особенно в коллекциях с тяжелым хвостом частот (много очень частотных и очень редких слов) – DFR (Divergence-from-randomness)

Полезные прикладные моменты для sparse

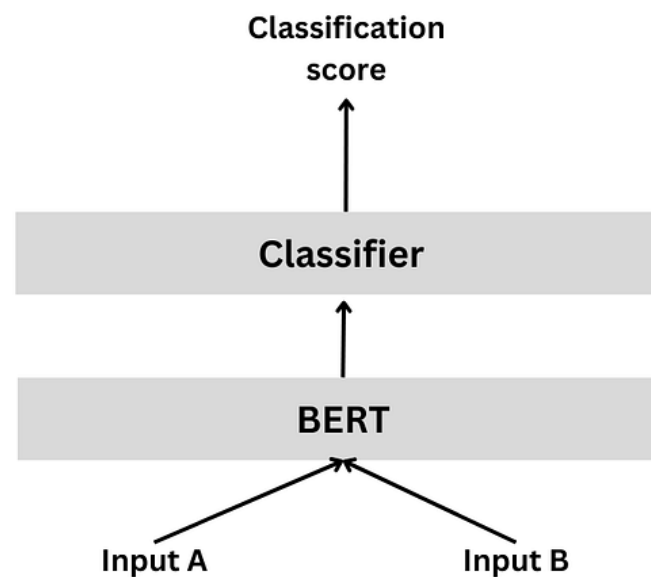
- Языкозависимая токенизация – разные языки требуют специфичных правил разбиения на токены
- Лемматизация & стемминг
- Character n-grams и поиск по частичным совпадениям – справляемся с опечатками
- Или через edit-distance
- Нормализация чисел и единиц измерения, и вообще всего
- Имеет смысл разносить куски текстов по полям
- Иерархия важности частей текста: title >> headings >> abstract >> body
- Поддержка словарей аббревиатур, доменных терминов

Dense-подходы

Bi-encoder



Cross Encoder



Bi-encoder

- Один вектор на запрос, один на документ
- Дешевая стадия отбора кандидатов из миллионов документов
- Возможность предвычисления индекса документов оффлайн
- Эффективное масштабирование через ANN поиск
- Размытие чанков - в длинных документах важная информация может "раствориться" в среднем представлении. Можно полечить
 - более короткими чанками
 - Hierarchical pooling (сначала pooling внутри чанков, потом между ними)

Cross-encoder

- Кодирует тексты совместно
- Медленный ($O(N \times M)$ для N запросов и M документов)
- Более точный, но требует онлайн-вычислений для каждой пары

В RAG используются для реранжирования выдачи от bi-encoder'ов

Симметричный vs асимметричный поиск

Симметричный:

- Запрос и документ одного типа/стиля (поиск дубликатов, матчинг, кластеризация).
- Векторы запроса и документа «живут» в одном пространстве. Сходство измеряет семантическую близость.

Асимметричный:

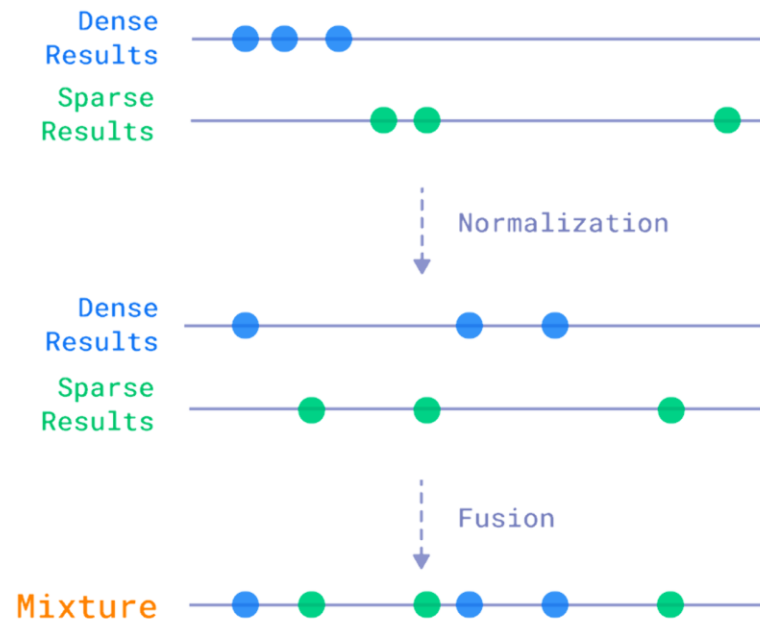
- Короткий запрос → длинный пассаж/документ.
- Векторы могут жить в скоррелированных подпространствах, и важно совпадение информации в ответе, а не общая тема.
- Типичные ошибки: использовать симметричную модель в RAG; ненастроенный чанкинг, неправильно подобранные метрики и меры близости

Мысли о мерах близости

- Симметричный поиск - cosine distance на L2-нормализованных векторах - база, потому что модель обучалась на угловое сходство, длины векторов не несут полезной информации, даже часто вредят.
- В ассиметричном поиске у некоторых моделей могут быть разные распределения норм для query/docs. Если модели важна информация о нормах - рабочий вариант dot product или cosine после явной L2-нормализации обеих сторон, если хотите убрать влияние норм и стабилизировать ранжирование. Выбор зависит от того, как обучалась модель и как построен индекс.
- L2 без нормализации чувствителен к нормам, при доменном сдвиге распределение норм меняется и ранжирование плавают
- Manhattan/chebyshev/корреляция и прочие метрики часто не согласованы с обучением - большинство эмбеддеров оптимизируют косинус или dot. Несовпадение обучение-инференс снижает качество

Гибридный поиск

Гибридный поиск объединяет сильные стороны классических (sparse) и плотных (dense) моделей, обеспечивая надёжность и точность на разных типах запросов.



Классический гибридный поиск

- Выполняются параллельно два поиска:
 - Sparse-поиск
 - Dense-поиск
- Объединение кандидатов: $C = C_{BM25} \cup C_{Dense}$.
- Нормализация скорингов каждого канала (z-score или Min-Max) для сопоставимости.
- Итоговый скоринг:
 - $\text{score}(d) = \alpha \cdot \hat{s}_{BM25}(d) + (1 - \alpha) \cdot \hat{s}_{Dense}(d)$,
 - где $\hat{s}$ — нормализованные значения, α настраивается на валидации.

Filter-then-hybrid

- Pre-filters (дата, язык и др.) применяются к индексу ДО выполнения основного поиска.
- Суженное пространство обрабатывается параллельно:
 - Sparse + Dense поиск по отфильтрованному набору.
- Late interaction (например, ColBERT-подход) или пересчёт скорингов только по объединённому множеству кандидатов.

Pre-filters vs Post-filters

Pre-filters применяются ДО самого поиска, на уровне индекса или ранжирования

Преимущества:

- Существенно уменьшают размер области поиска, увеличивают скорость и масштабируемость.
- Могут использоваться структуры индекса (bitmap, inverted lists).

Недостатки:

- Ограничивают recall: слишком жёсткие фильтры могут отбросить релевантные документы.
- Стоимость сложных предикатов на уровне индекса (например, диапазонные запросы по дате).

Post-filters применяются ПОСЛЕ первичного поиска и ранжирования - логически проще интегрировать, можно отфильтровать любой атрибут.

Преимущества:

- Не влияют на внутреннюю структуру индекса.
- Гибкость: легко изменить бизнес-логику без реиндексации.

Недостатки:

- Дополнительная задержка: сначала нужно получить и ранжировать полный набор.
- Может привести к «дыркам» в выдаче, если топ-K отображены без учёта фильтра, доступных для фильтрации может не хватить для заполнения выдачи.

Sparse+Dense на первом этапе + late interaction

- Сначала комбинированный поиск: быстрый Sparse + ANN-Dense для отбора топ-K.
- Затем для полученных K документов:
 - Применяется более тяжёлая модель с late interaction, учитывающая точное выравнивание токенов.
- Обмен данными между каналами происходит на этапе re-ranking.

Фьюжн скоринги - Reciprocal Rank Fusion (RRF)

$$RRFscore(d \in D) = \sum_{r \in R} \frac{1}{k+r(d)}$$

- Плюсы: стабильность при появлении новых документов и доменных сдвигах.
- Минусы: не учитывает абсолютную разницу скорингов.

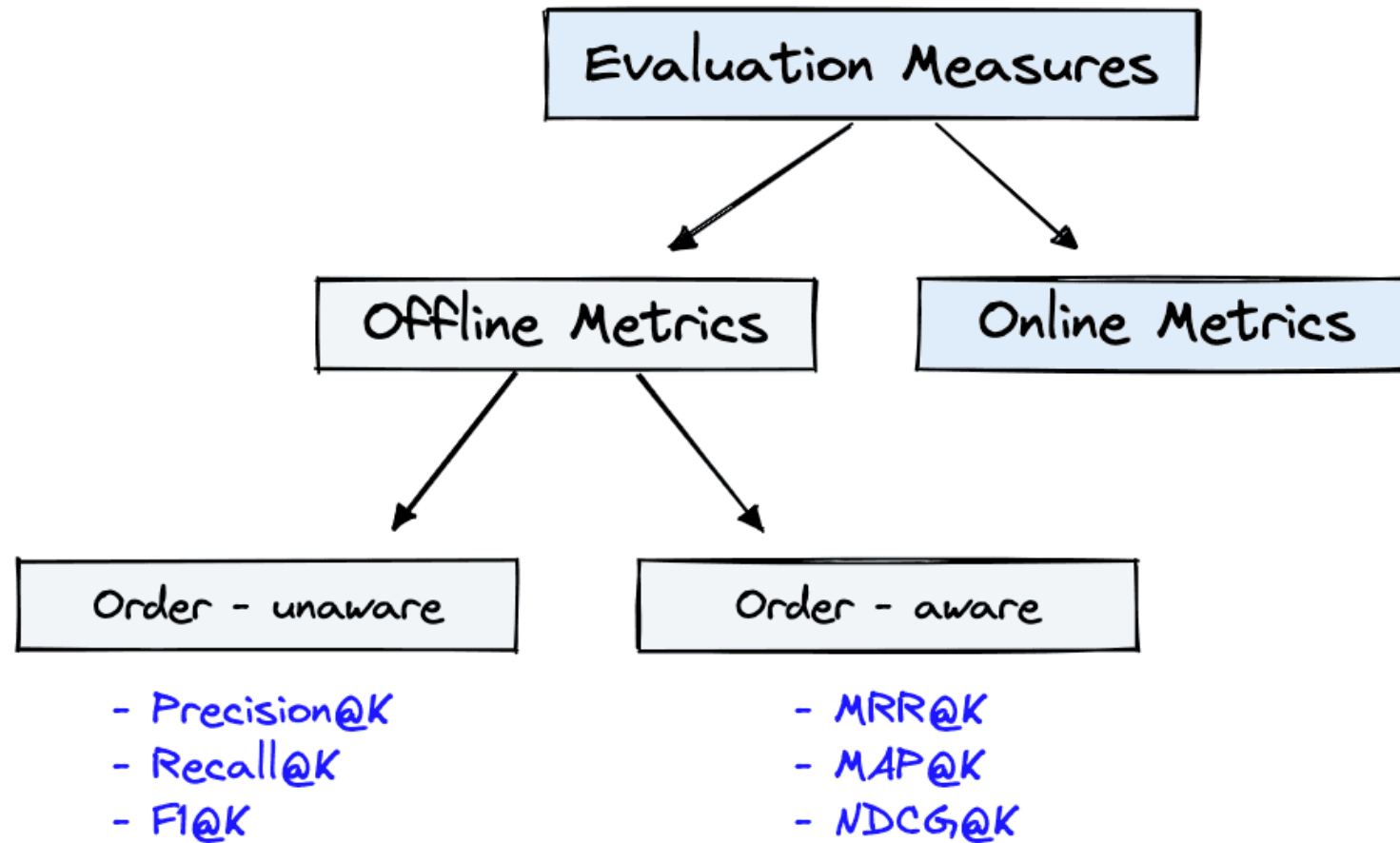
Взвешенная сумма после нормализации

- Нормализация скорингов каждого канала – z-score, min-max
- Итоговый скоринг – взвешенная сумма
- Веса определяются на валидационной выборке через grid search или bayesian optimization
-

Многостадийный поиск

- Stage 1: Генерация кандидатов. На первом этапе задача — быстро отобрать из пула документов небольшое множество кандидатов, которые далее пройдут более тщательную обработку.
- Stage 2: Лёгкая повторная калибровка без тяжёлых Cross-encoder моделей.
Нормализация скорингов, бизнес-бусты, диверсификация (MMR), дедупликация и т.п.
- Stage 3: ре-ранжирование (опционально)

Метрики Information Retrieval

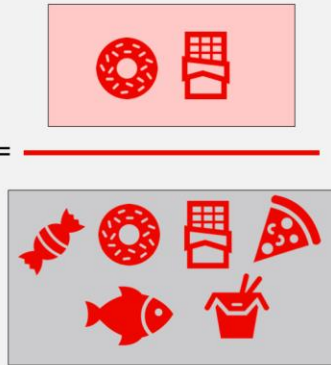


Precision@k

Interaction results

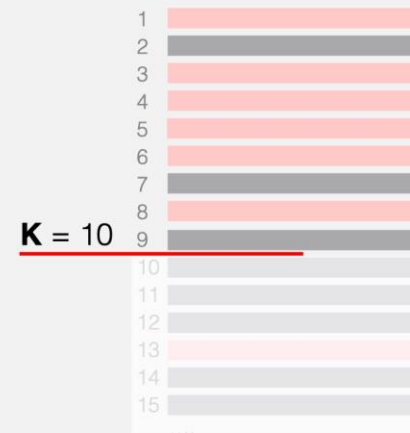


Precision @6 = $\frac{2}{6} = 0.33$



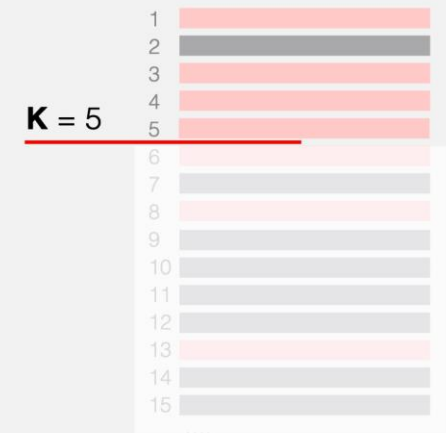
Not relevant Relevant

Precision@10 = 0.6

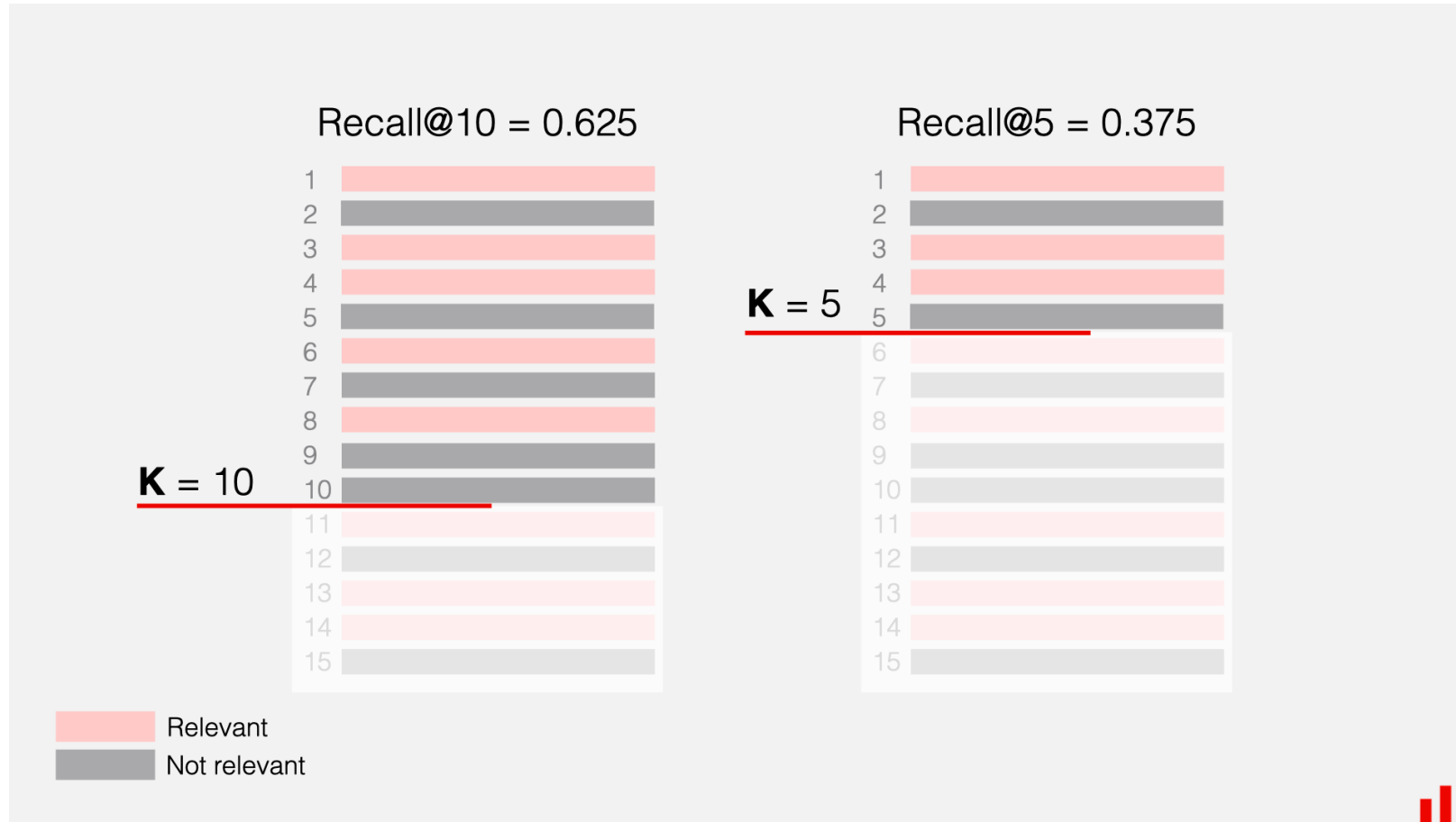


Not relevant Relevant

Precision@5 = 0.8



Recall@k

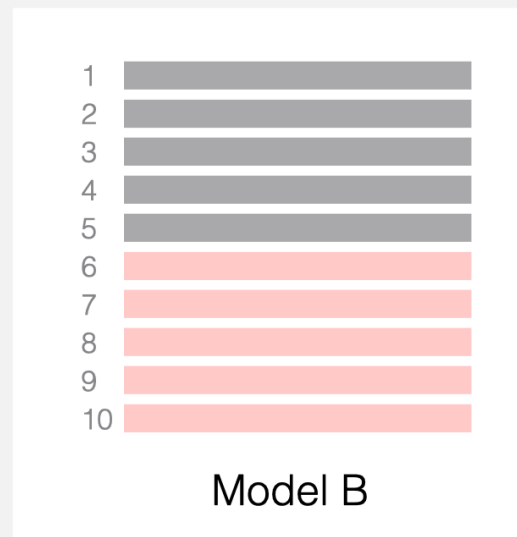


Precision & recall резюме

- Precision и recall @k помогают оценить качество рекомендательных и ранжирующих систем.
- Precision @k определяет, сколько элементов среди выбранных k являются релевантными.
- Recall @k измеряет долю релевантных элементов, выбранных в top k.
- Также можно использовать F-меру для получения сбалансированной оценки precision и recall @k.
- Precision, recall и F-мера могут принимать значения от 0 до 1. Более высокие значения соответствуют лучшему качеству.
- Однако precision и recall отражают только количество релевантных элементов @k, не оценивая качество ранжирования внутри списка.

Проблема с оценкой ранжирования

Precision@10 = 50%

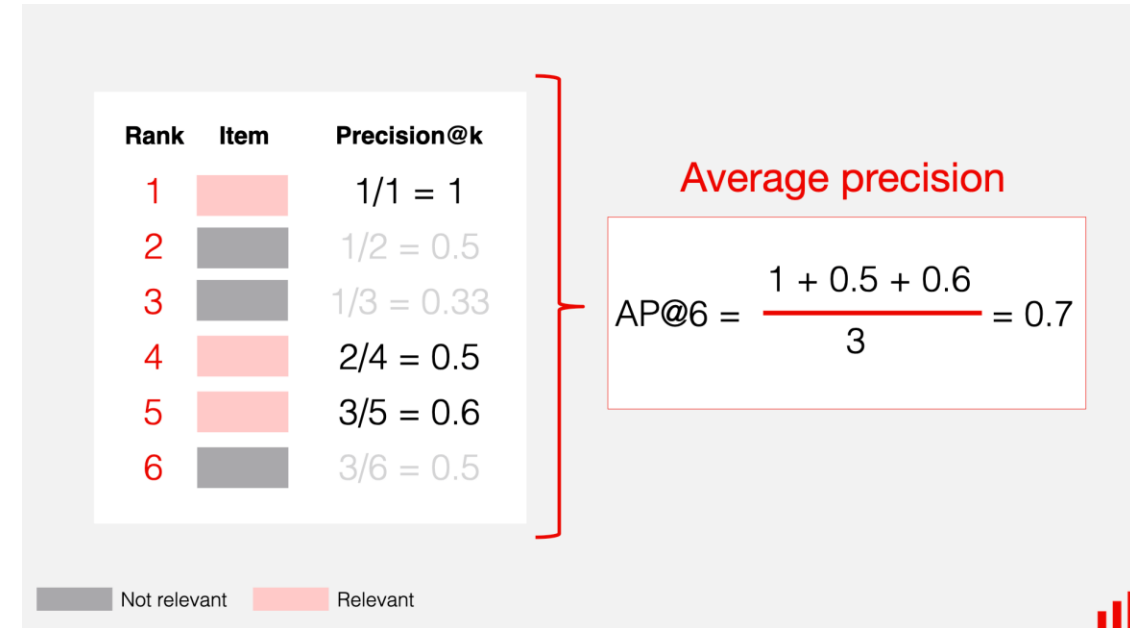


Relevant
Not relevant



Average Precision

- Шаг 1. Сначала необходимо узнать общее количество релевантных элементов в результатах поиска @k. Это будет N.
- Шаг 2. Затем необходимо определить позиции каждого релевантного элемента в K и вычислить точность для каждой из этих позиций. Вычисление точности происходит как обычно: количество релевантных элементов делится на общее количество элементов до этой позиции. Есть одно замечание: мы учитываем точность только для рангов, где элементы релевантны. Мы пропускаем все позиции, где элементы нерелевантны.
- Шаг 3. Наконец, мы суммируем значения точности в каждой соответствующей позиции и делим сумму на общее количество релевантных элементов.



$$AP@K = \frac{1}{N} \sum_{k=1}^K Precision(k) \times rel(k)$$

Разные варианты AP

		Precision @k
1	Relevant	1
2	Relevant	1
3	Relevant	1
4	Not relevant	0.75
5	Not relevant	0.6
6	Not relevant	0.5

$$AP = 3/3 = 1$$

		Precision @k
1	Relevant	1
2	Relevant	1
3	Not relevant	0.67
4	Not relevant	0.5
5	Not relevant	0.4
6	Relevant	0.5

$$AP = 2.5/3 = 0.83$$

		Precision @k
1	Not relevant	0
2	Not relevant	0
3	Not relevant	0
4	Relevant	0.25
5	Relevant	0.4
6	Relevant	0.5

$$AP = 1.15/3 = 0.38$$

Not relevant Relevant



Mean Average Precision

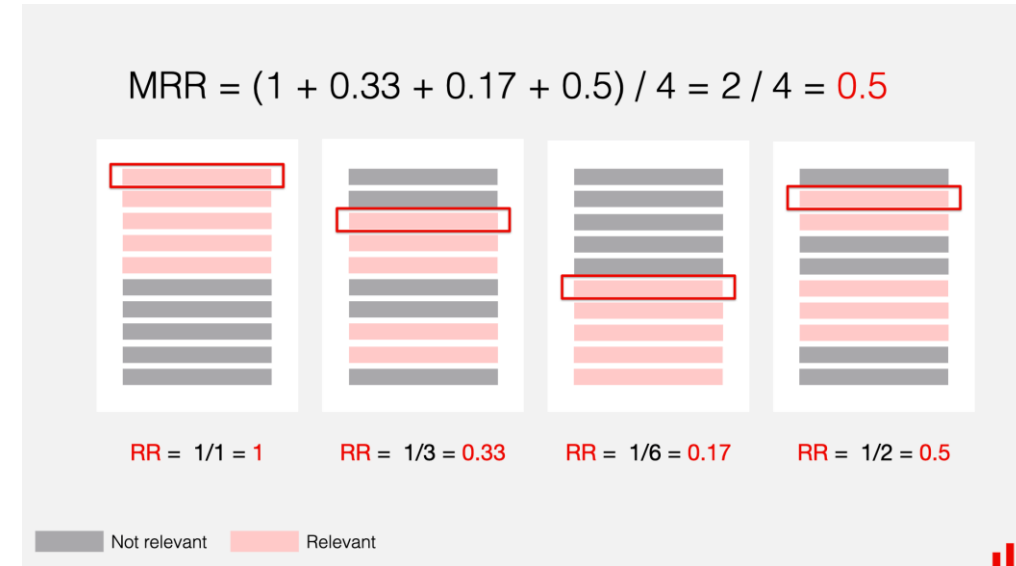
- Mean Average Precision (MAP) — это метрика качества ранжирования. Она учитывает количество релевантных рекомендаций и их положение в списке.
- MAP @k рассчитывается как среднее арифметическое значение Average Precision (AP) @k по всем запросам.
- MAP может принимать значения от 0 до 1, где 1 соответствует идеальному ранжированию, когда все соответствующие элементы находятся наверху. Более высокие значения MAP означают лучшее качество.

$$\text{MAP@K} = \frac{1}{U} \sum_{u=1}^U \text{AP@K}_u$$



Mean Reciprocal Rank (MRR)

- Mean Reciprocal Rank (MRR) — это метрика качества ранжирования. Она учитывает позицию первого релевантного элемента в ранжированном списке.
- Можно рассчитать MRR как среднее значение обратных рангов по всем запросам.
- Обратный ранг — это величина, обратная положению первого релевантного элемента. Если первый релевантный элемент находится на позиции 2, обратный ранг равен $1/2$.
- Значения MRR находятся в диапазоне от 0 до 1, где «1» указывает на то, что первый релевантный элемент всегда находится наверху.
- Фактически $MRR@k$ оценивает, насколько быстро система может отобразить первый релевантный элемент в топ-K результатов.
- Более высокий MRR означает лучшее качество системы.



$$MRR = \frac{1}{U} \sum_{u=1}^U \frac{1}{rank_i}$$

NDCG

- Normalized Discounted Cumulative Gain (NDCG) — это метрика качества ранжирования. Она сравнивает ранжирование с идеальным порядком, в котором все релевантные элементы находятся в верхней части списка.
- NDCG@k определяется путем деления Discounted Cumulative Gain (DCG) на идеальный DCG, представляющий собой идеальный рейтинг.
- DCG измеряет общую релевантность элементов в списке с дисконтированием, которое помогает компенсировать уменьшение "ценности" товаров, расположенных ниже в списке.
- Можно агрегировать NDCG по всем запросам, чтобы получить общую оценку качества ранжирования системы.
- NDCG может принимать значения от 0 до 1, где 1 указывает на соответствие идеальному порядку, а более низкие значения представляют более низкое качество ранжирования.

MAP

$$\text{MAP@K} = \frac{1}{U} \sum_{u=1}^U \text{AP@K}_u$$

NDCG

$$\text{DCG@K} = \sum_{k=1}^K \frac{rel_i}{\log_2(i+1)}$$

$$\text{NDCG@K} = \frac{\text{DCG@K}}{\text{IDCG@K}}$$

$$\begin{aligned} k=1: & \frac{1}{\log_2(1+1)} = \frac{1}{\log_2(2)} = \frac{1}{1} = 1 \\ k=2: & \frac{1}{\log_2(1+2)} = \frac{1}{\log_2(3)} \approx \frac{1}{1.585} \approx 0.63 \\ k=3: & \frac{1}{\log_2(1+3)} = \frac{1}{\log_2(4)} = \frac{1}{2} = 0.5 \\ k=4: & \frac{1}{\log_2(1+4)} \approx \frac{1}{\log_2(5)} \approx \frac{1}{2.322} \approx 0.43 \\ k=5: & \frac{1}{\log_2(1+5)} \approx \frac{1}{\log_2(6)} \approx \frac{1}{2.585} \approx 0.39 \end{aligned}$$

Метрики по этапам RAG-pipeline

Stage 1: Retrieval

- Recall@k: базовая метрика полноты.
- p95/p99 latency: 95- и 99-перцентили latency запроса — важны для систем реального времени.
- Анализ компромисса между latency и Recall@k при разных конфигурациях (размер индекса, алгоритм ANN, параметры поиска).

После лёгкой калибровки/фьюжна

- NDCG@k - оценка улучшенного ранжирования после объединения сигналов (семантика+BM25).
- MRR - стабильность ранга первого релевантного документа.
- Стабильность рангов в целом измеряется ранговой корреляцией (Спирмена или Кендала) между исходным и откалиброванным списком.